

EDA using Univariate Analysis

100 Days of Machine Learning | Day 20 | CampusX

1. What is Exploratory Data Analysis (EDA)?

The core purpose of EDA is to understand your data inside-out. Before applying machine learning algorithms, you must know how your variables behave. The most basic and important step in EDA is **Univariate Analysis**.

- **Univariate Analysis:** Analyzing one single column (variable) at a time independently.
- **Bivariate Analysis:** Analyzing two columns simultaneously to find relationships between them.
- **Multivariate Analysis:** Analyzing more than two columns simultaneously.

2. Understanding Data Types

Before plotting any graphs, you must ask one crucial question about your column: *What kind of data does it hold?* The nature of the plot depends entirely on this.

- **Categorical Data:** Data that represents categories or groups. (e.g., Nationality, Gender, College Branch, Survived (Yes/No), Passenger Class (1,2,3)).
- **Numerical Data:** Data that represents measurable quantities or numbers. (e.g., Age, Height, Weight, Ticket Price).

3. Univariate Analysis on Categorical Data

For categorical data, your main goal is to find out the frequency of each category (how many times each category appears).

Countplot (Using Seaborn)

A countplot creates a bar chart showing the frequency of each category.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Creates a bar chart showing counts of 'Survived' (0 and 1)
sns.countplot(df['Survived'])
plt.show()

# Alternatively using pandas:
# df['Survived'].value_counts().plot(kind='bar')
```

Pie Chart (Using Pandas/Matplotlib)

If you want to view the distribution in terms of percentages rather than raw counts, use a pie chart.

```
# autopct='%0.1f%%' displays the percentages on the chart
df['Survived'].value_counts().plot(kind='pie', autopct='%0.1f%%')
plt.show()
```

4. Univariate Analysis on Numerical Data

For numerical (continuous) data, the goal is to understand the *distribution* of the numbers (e.g., are most people young or old? Are there any outliers?).

Histogram

A histogram groups numerical data into "bins" (ranges) and plots the count of values falling into each bin.

```
# Using matplotlib
plt.hist(df['Age'], bins=50) # 'bins' parameter controls the number of intervals
plt.show()
```

Distplot (KDE - Kernel Density Estimation)

A distplot is an improvement over a histogram. It plots a smooth probability density curve (PDF - Probability Density Function) over the histogram. The Y-axis represents the probability density.

```
# Using seaborn
sns.distplot(df['Age'])
plt.show()
```

Use case: It helps you quickly identify if your data follows a Normal Distribution or if it is skewed (left-skewed/right-skewed).

Boxplot

A boxplot gives a visual representation of the **5-Number Summary** of your data. It is the best tool for identifying **Outliers**.

- **Median (50th Percentile):** The middle value.
- **Q1 (25th Percentile):** 25% of the data falls below this line.
- **Q3 (75th Percentile):** 75% of the data falls below this line.
- **IQR (Interquartile Range):** $Q3 - Q1$.
- **Minimum:** Calculated as $Q1 - 1.5 * IQR$
- **Maximum:** Calculated as $Q3 + 1.5 * IQR$

Any data points lying outside the calculated minimum and maximum are plotted as individual dots and are considered *potential outliers*.

```
# Boxplot to check for outliers in ticket prices
sns.boxplot(df['Fare'])
plt.show()
```

5. Statistical Methods for Numerical Data

Apart from graphs, you can also use pandas built-in statistical functions to analyze single numerical columns:

```
print(df['Age'].min())    # Minimum value
print(df['Age'].max())    # Maximum value
print(df['Age'].mean())   # Average value
print(df['Age'].skew())   # Checks for skewness (0 = perfectly symmetrical)
```

Skewness: If the `skew()` value is perfectly 0, it's symmetrical (Normal Distribution). Positive values indicate right skew, and negative values indicate left skew.